Developing An Intelligent Chatbot System

Task 1: Finding the cheapest train ticket

To help their customers in finding the cheapest available ticket for their intended journey in the UK

Introduction

- This project focuses on developing a conversational chatbot that helps users search for train tickets.
- The chatbot collects journey information through a step-by-step dialogue and then returns a suitable ticket option with a booking link.
- The system was designed for users who may not want to manually search through train websites and instead prefer a guided conversation.

What the Chatbot Is Designed to Do??

- The main objective of the first task was to build a chatbot that can help users find the cheapest available train ticket for their journey.
- To do this, the chatbot needs to understand the user's travel request, collect missing information such as departure station, destination station, travel date, time, and ticket type, and then produce a ticket result.
- The chatbot also supports extra journey details such as return tickets and railcards. This makes the conversation more realistic because many passengers need discounts or return journeys when booking train tickets.

Development Approach: How the System Was Built?

- The chatbot was developed as a FastAPI web application. The backend handles the conversation logic, ticket search process, station validation, and API communication. The frontend provides a simple chat interface where the user can type messages and receive chatbot responses.

- The system was divided into separate Python files so each part had a clear responsibility. For example, conversation.py manages the dialogue, nlu.py detects intent and extracts journey details, ticket_api.py connects to the National Rail API, and services.py handles ticket search and fare calculation.

Conversation Design : Collecting Journey Details Step by Step

- The chatbot uses a slot-filling approach. This means it checks which information has already been collected and asks for anything missing. For example, if the user only says they want a ticket, the bot asks for the departure station. After that, it asks for the destination, date and time, ticket type, and railcard information.

- This approach makes the chatbot easier to use because the user does not need to provide every detail in one sentence. The conversation continues naturally until enough information has been collected to search for a ticket

Ticket Search and API Integration : Using National Rail Data Where Available

- The chatbot attempts to use the National Rail journey planner API to retrieve journey information. The API request includes the origin station, destination station, travel time, passenger details, and railcard information where available.

- When the API returns useful journey details, such as departure time, arrival time, platform, duration, or price, the chatbot uses those values. However, API responses may not always include complete fare or platform information. To handle this, the system includes fallback logic so the chatbot can still complete the conversation.

Fare Handling : Combining API Data with Fallback Fare Logic

- One challenge during development was that the API did not always return ticket prices. To avoid the chatbot failing or showing no result, a fallback fare engine was implemented. If the API returns a real price, the chatbot uses that price. If the API does not return a price, the chatbot uses a deterministic estimated fare.

- This fallback keeps the chatbot functional during testing and demonstration. It also makes the behaviour more reliable because the user still receives a ticket result even when the live API response is incomplete.

Railcard Discount Implementation : Adding Discount Support

- A railcard feature was added so users can apply discounts to their ticket. The chatbot first asks whether the user has a railcard. If the user says yes, it asks which railcard they want to use. It supports common UK railcards such as 16-25, Senior, Disabled Persons, Family & Friends, Two Together, Network, and Veterans.

- During testing, an issue occurred where typing 16-25 was incorrectly treated as a journey time or date. This was fixed by making the railcard collection state-aware. When the bot is asking for a railcard, the system now interprets the user’s answer only as a railcard response, not as travel time information.

Station Validation : Improving Station Name Accuracy

- Station names are loaded from a station CSV file. During testing, the chatbot sometimes selected non-passenger railway locations, such as sidings, or chose the wrong station when a name had multiple matches. For example, a user typing Oxford could be matched with Manchester Oxford Road.

- To improve this, station matching was refined. Exact station names are now prioritised over partial matches. Non-passenger locations such as sidings, depots, and internal rail locations are filtered out. This makes the chatbot more suitable for passenger ticket searches.

Return Ticket Support : Handling Outward and Return Journeys

- The chatbot was extended to support return tickets. If the user says they want a return ticket, the bot asks for the return date and time. The final response then displays separate outward and return journey sections.

- This makes the chatbot match the task requirements more closely, because the task example includes journeys where the passenger travels out and comes back later. The chatbot now collects both outward and return journey details before presenting the ticket result.

Error Handling and Fallbacks : Making the Chatbot More Reliable

- Several fallback mechanisms were added to make the chatbot more reliable. If the API does not return platform details, the chatbot displays “Platform: To be announced” instead of showing fake platform numbers. If the API does not return a price, the chatbot uses the fallback fare. If the user enters an invalid station, the chatbot asks for a valid UK station name.

- These improvements make the chatbot more professional because it avoids displaying misleading information where possible.

Testing and Improvements : How the Chatbot Was Tested

- The chatbot was tested using common journey examples such as Norwich to London, Norwich to Oxford, and return journeys. Railcard scenarios were also tested to confirm that discounts were applied correctly.

- Testing helped identify several issues, including incorrect railcard parsing, wrong station matching, fixed mock platform numbers, and missing return journey handling. Each issue was fixed step by step, improving the chatbot’s accuracy and usability.

Final Outcome : What the Chatbot Can Do Now

- The final chatbot can collect journey details through conversation, validate station names, support single and return tickets, apply railcard discounts, use API journey data when available, and provide a ticket result with a booking link.

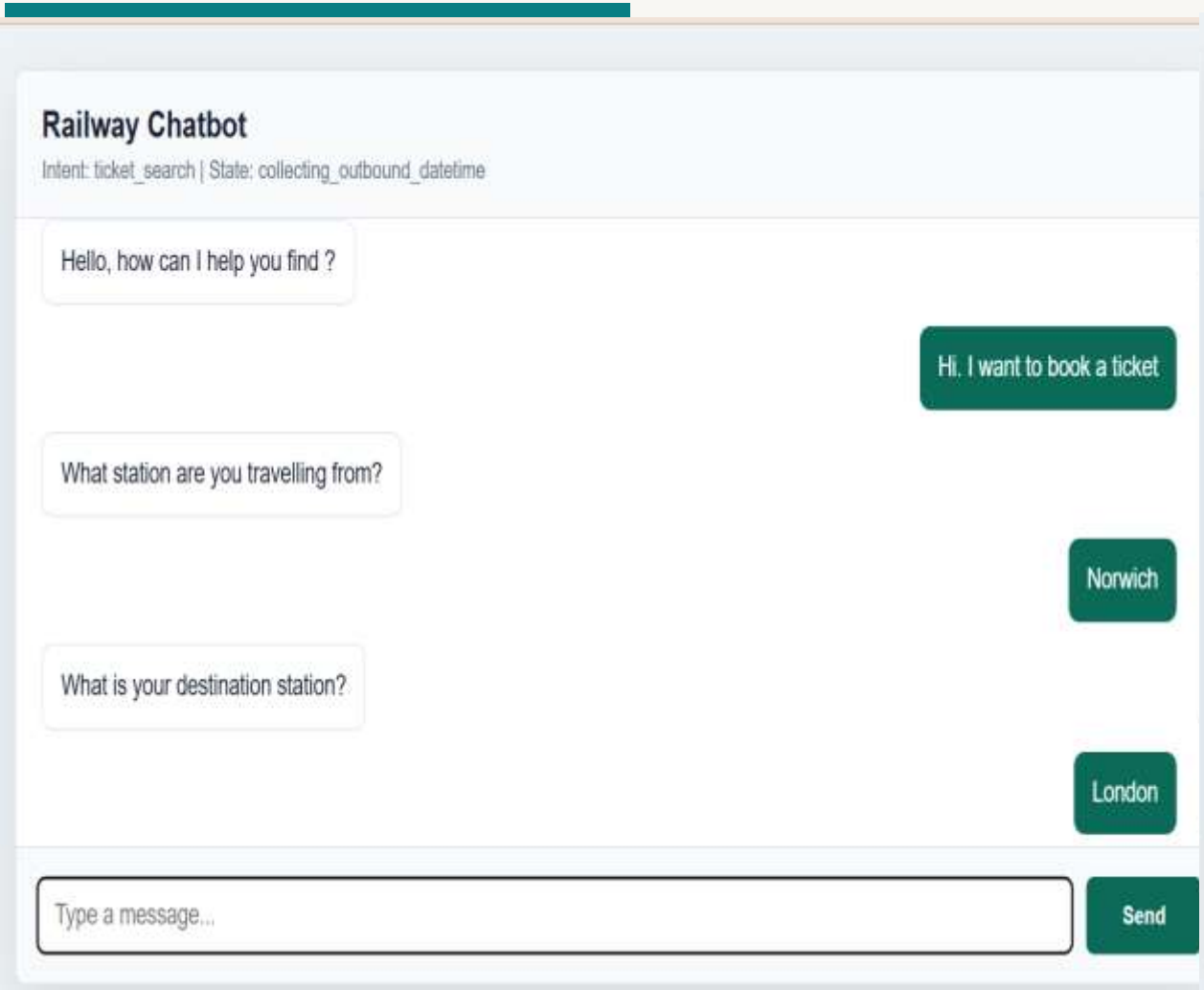
- Although some fallback logic is still used when API data is incomplete, the chatbot meets the main objective of guiding the user through the ticket search process and presenting a suitable train ticket option.

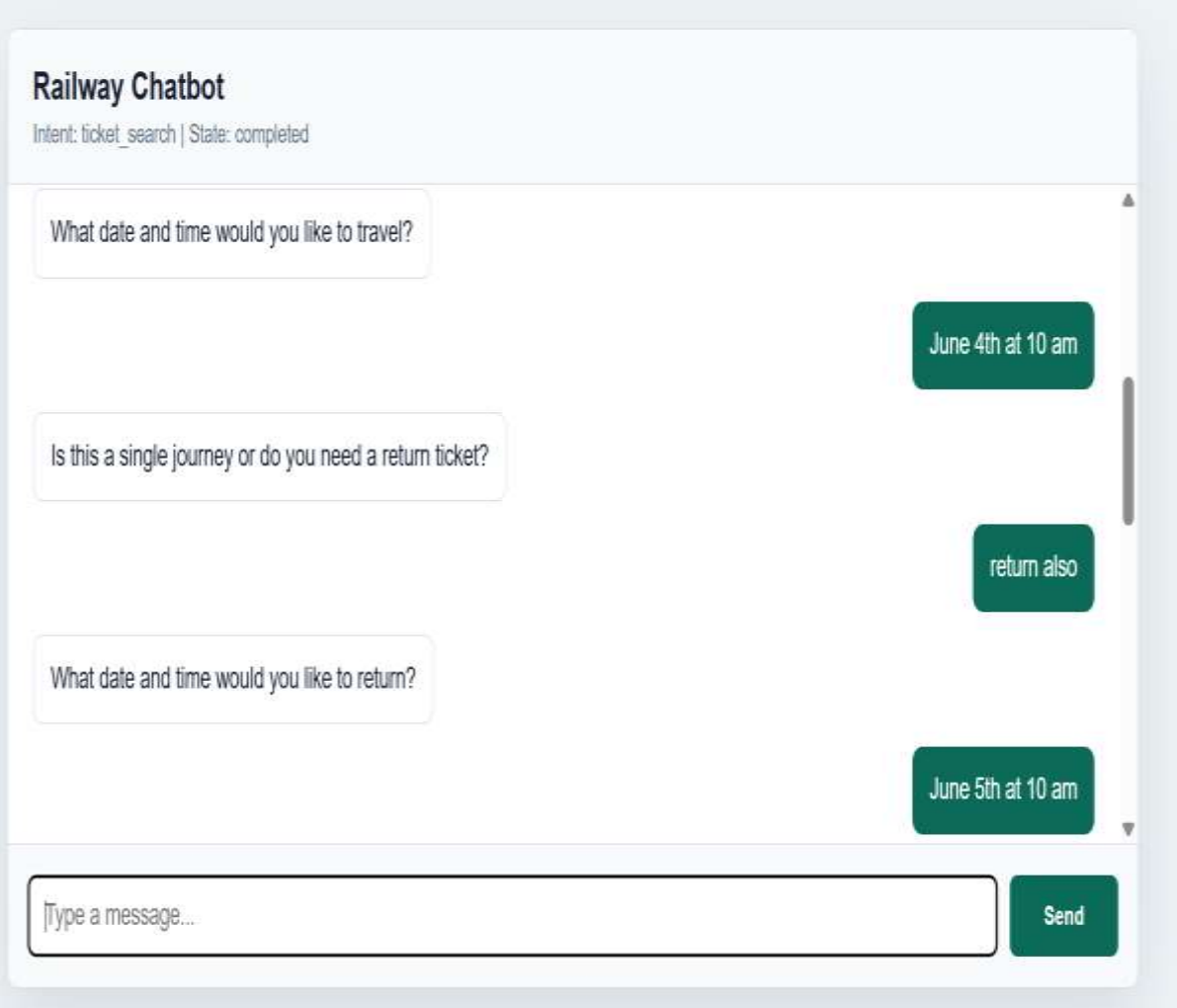
Limitations : Current System Limitations

- The main limitation is that the API does not always return complete live fare, platform, or duration information. Because of this, fallback data is sometimes used. The chatbot also currently presents one ticket option rather than comparing multiple live ticket providers.

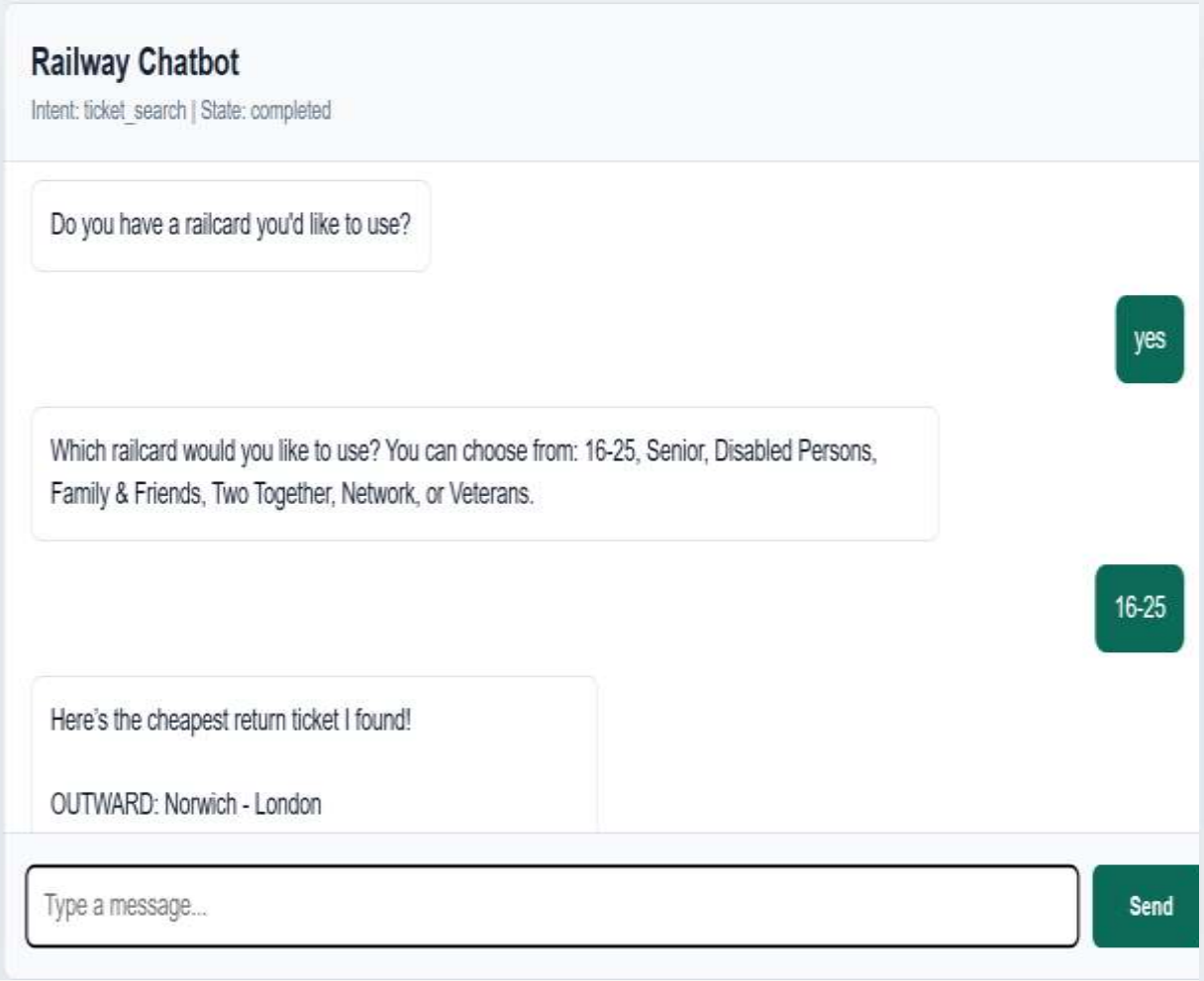
- In a future version, the chatbot could be improved by showing multiple ticket options, generating more accurate booking links, supporting before or after departure preferences, and improving real-time journey data handling.

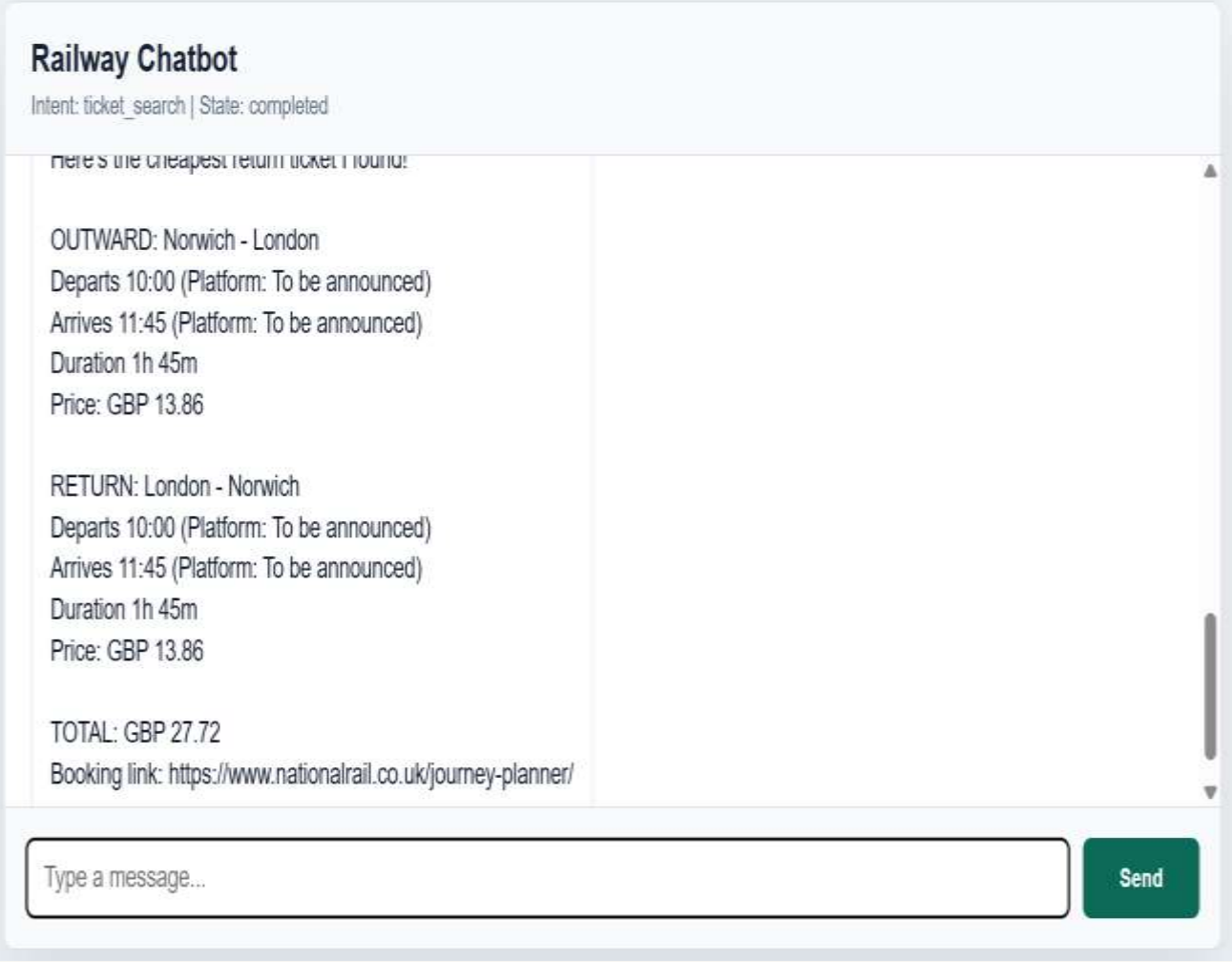
OUTPUT


Railway Chatbot
Intent: ticket_search | State: collecting_outbound_datetime
Hello, how can I help you find ?
Hi. I want to book a ticket
What station are you travelling from?
Norwich
What is your destination station?
London
Type a message...
Send


Railway Chatbot
Intent: ticket_search | State: completed
What date and time would you like to travel?
June 4th at 10 am
Is this a single journey or do you need a return ticket?
return also
What date and time would you like to return?
June 5th at 10 am
Type a message...
Send

OUTPUT


Railway Chatbot
Intent: ticket_search | State: completed
Do you have a railcard you'd like to use?
yes
Which railcard would you like to use? You can choose from: 16-25, Senior, Disabled Persons, Family & Friends, Two Together, Network, or Veterans.
16-25
Here's the cheapest return ticket I found!
OUTWARD: Norwich - London
Type a message...
Send


Railway Chatbot
Intent: ticket_search | State: completed
OUTWARD: Norwich - London
Departs 10:00 (Platform: To be announced)
Arrives 11:45 (Platform: To be announced)
Duration 1h 45m
Price: GBP 13.86
RETURN: London - Norwich
Departs 10:00 (Platform: To be announced)
Arrives 11:45 (Platform: To be announced)
Duration 1h 45m
Price: GBP 13.86
TOTAL: GBP 27.72
Booking link: https://www.nationalrail.co.uk/journey-planner/
Type a message...
Send

Task 2: Improving Customer Service

Predict the estimated arrival time of a delayed train

PURPOSE

- ➔ Goal: predict the estimated arrival time of a delayed train
- ➔ Scenario: passenger on Weymouth to London Waterloo (South Western Railway) service
- ➔ Train is delayed at an intermediate station (Southampton)
- ➔ Chatbot collects journey info and returns a predicted arrival time

Data Preprocessing

- ➔ Historical train performance data from South Western Railway (WAT↔WEY route)
- ➔ Two direction-specific datasets: WAT to WEY and WEY to WAT
- ➔ Data split: 70% training / 15% validation / 15% testing for hyperparameter tuning
- ➔ 80/20 for final model

Model Training

Three algorithms were trained and compared.

Random Forest

- ➔ Ensemble of decision trees
- ➔ Robust to noise
- ➔ n_estimators = [50, 100, 200]
- ➔ max_depth = [5, 10, 20, None]

XGBoost

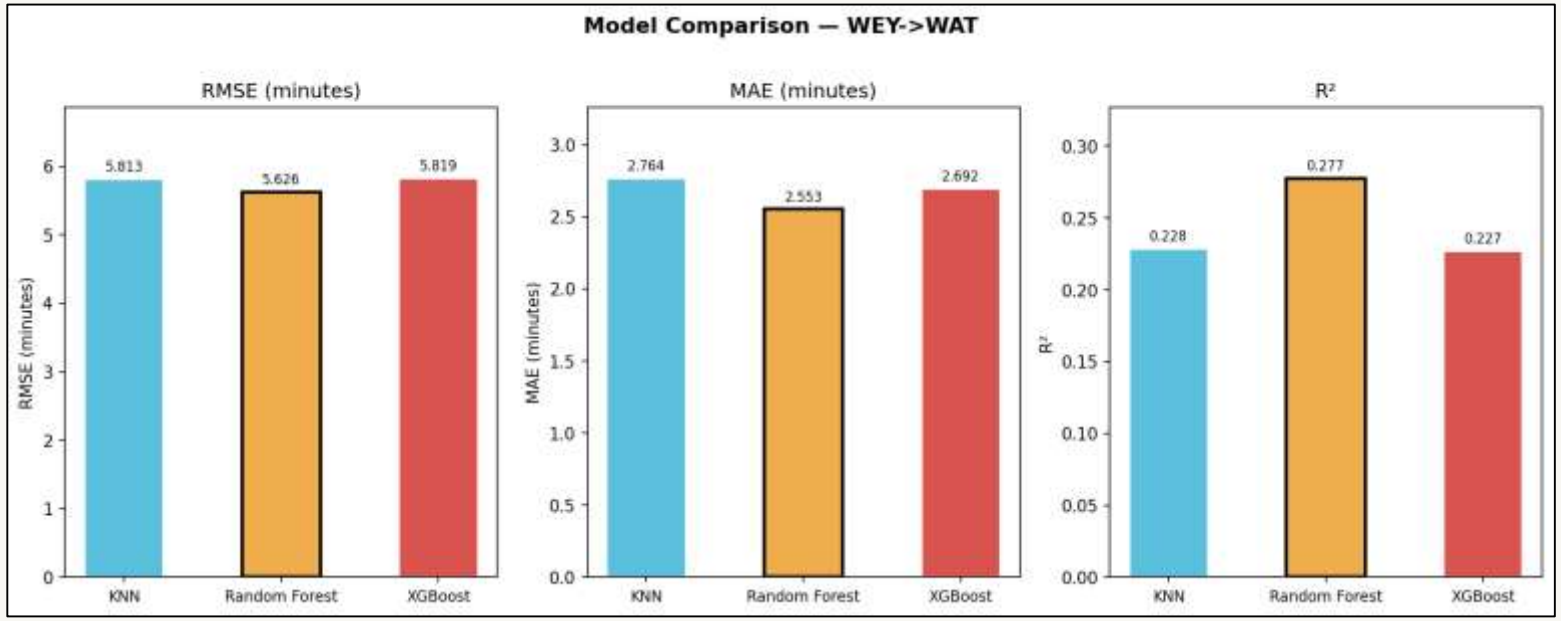
- ➔ gradient boosted trees
- ➔ often stronger on tabular data
- ➔ n_estimators = [100, 200, 300]
- ➔ max_depth = [3, 5, 7]
- ➔ learning_rate = 0.1

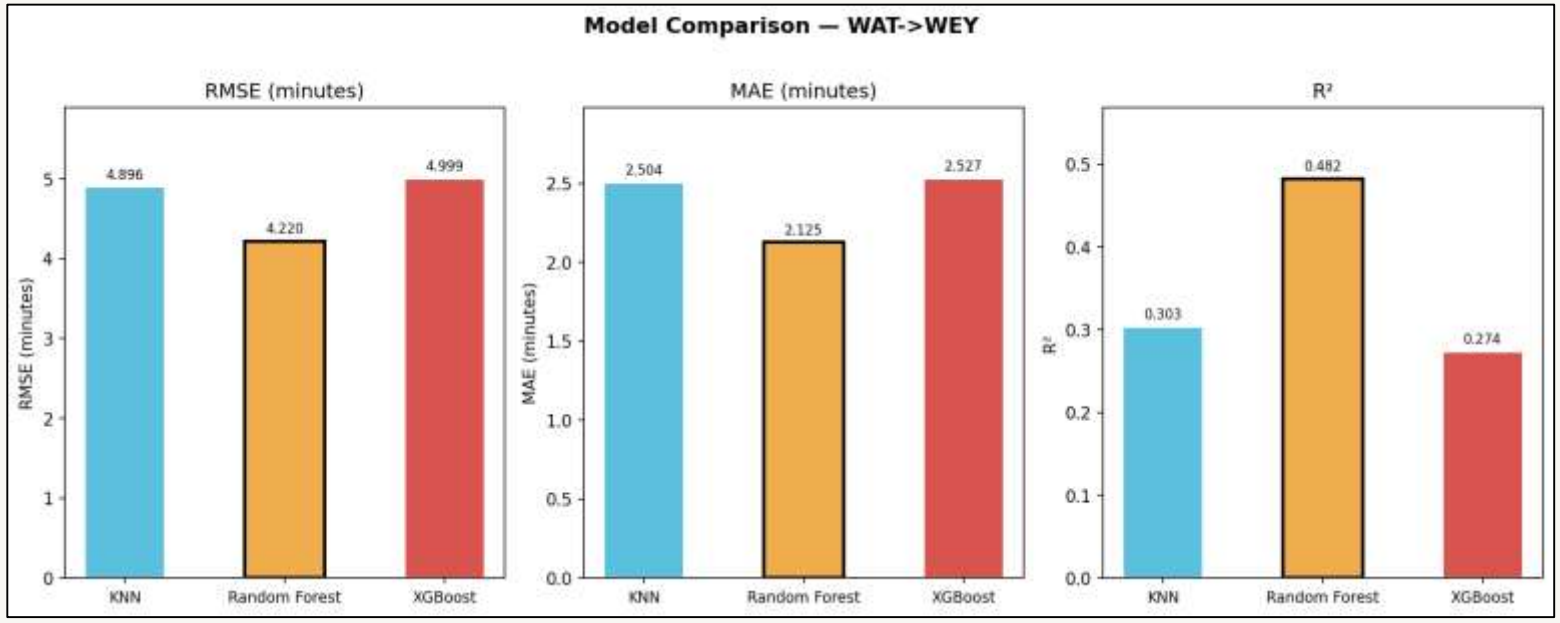
KNN Regressor

- ➔ predicts based on k nearest historical examples
- ➔ tuned over k values [3,5,10,15,20]

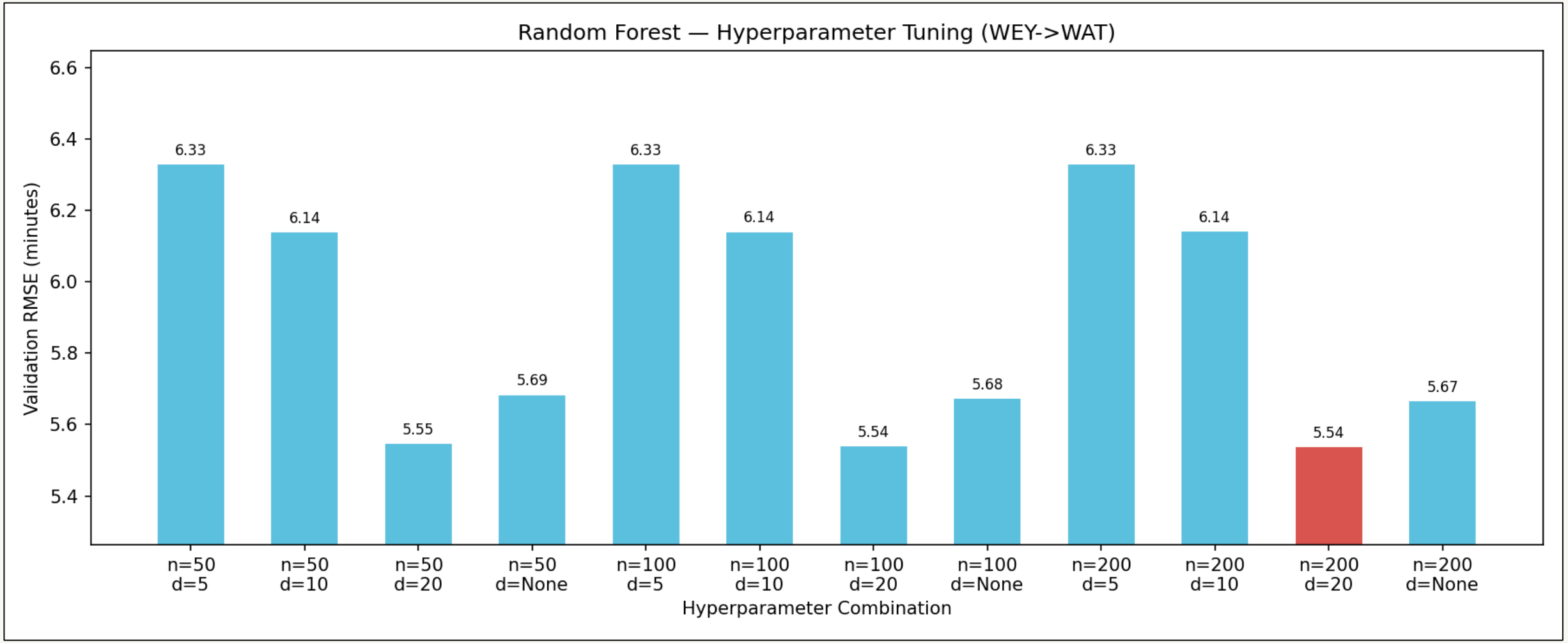
Best config per model selected by lowest validation RMSE

Model Comparison

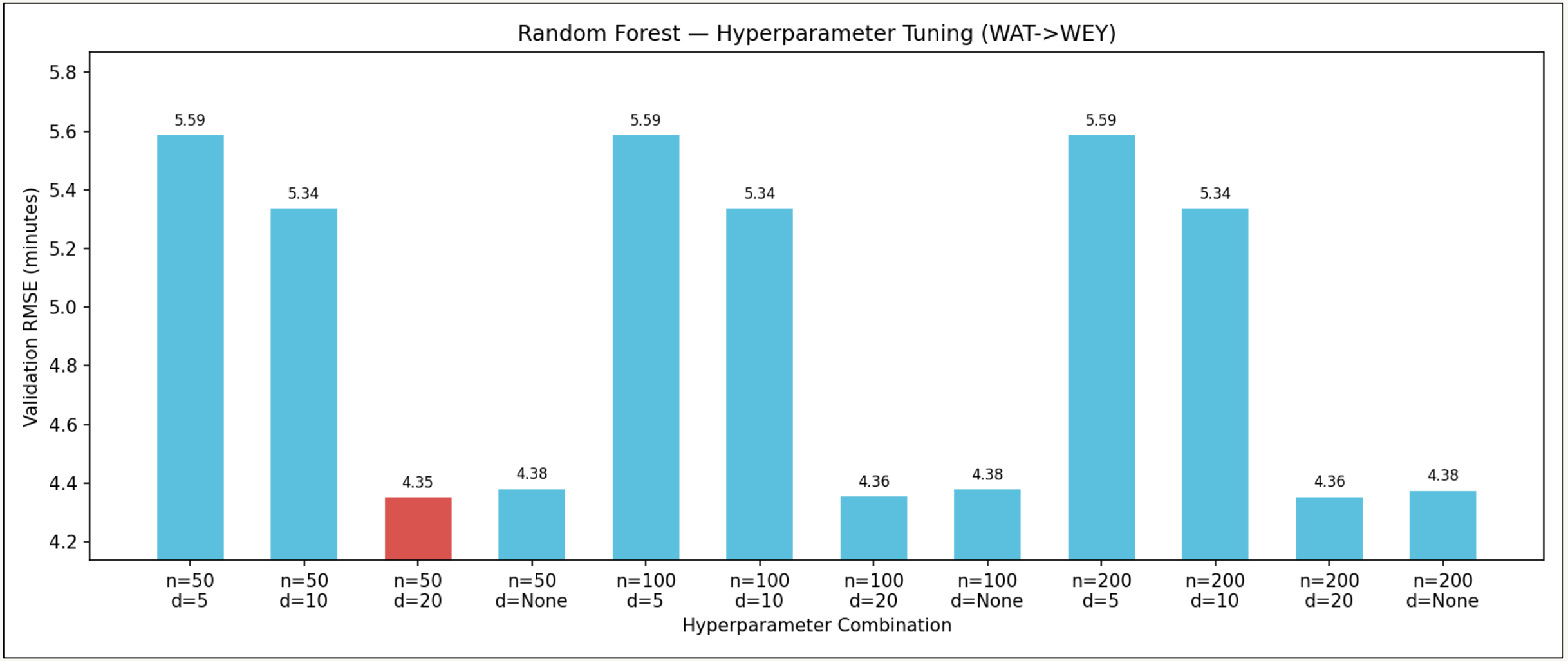




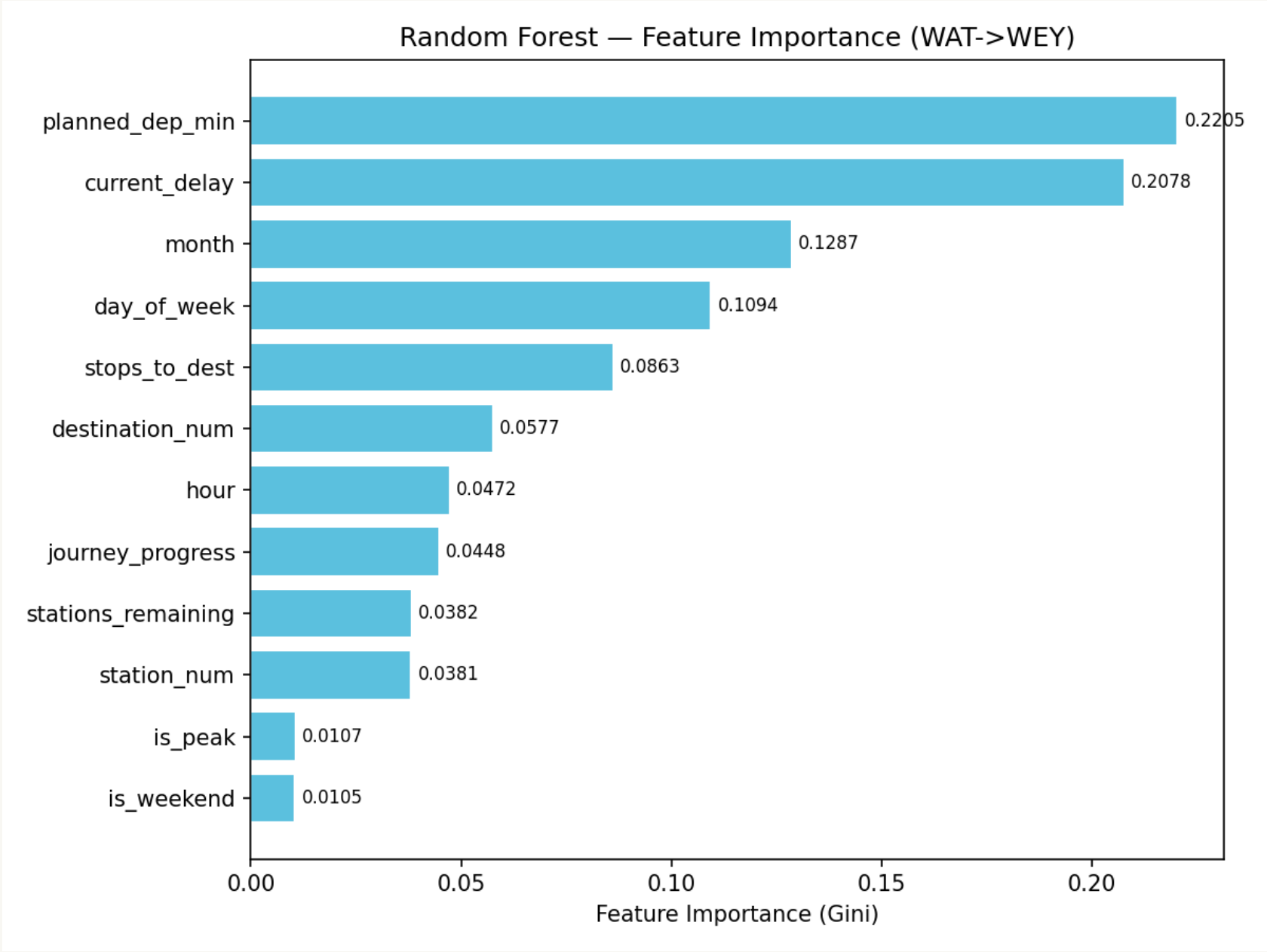
Hyperparameter Tuning Random Forest (WEY -> WAT)



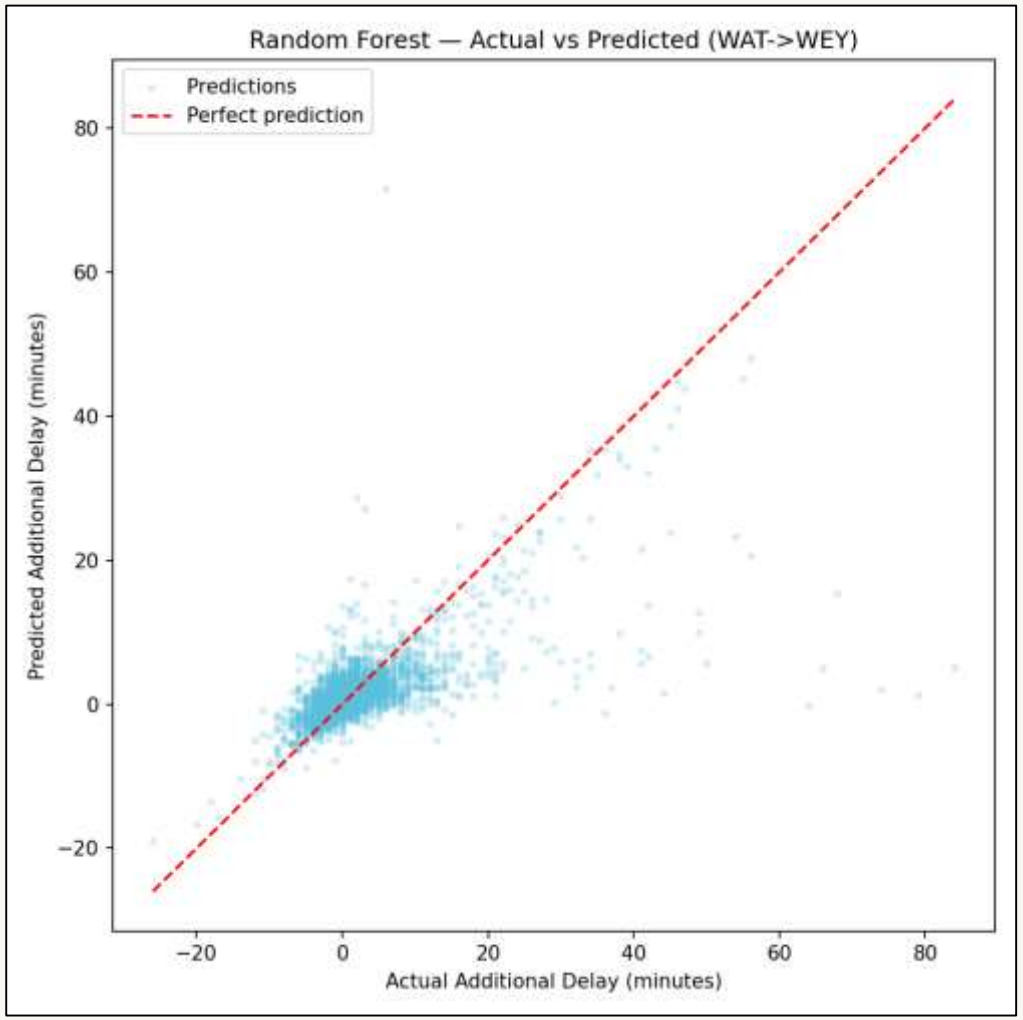
Hyperparameter Tuning Random Forest (WAT -> WEY)

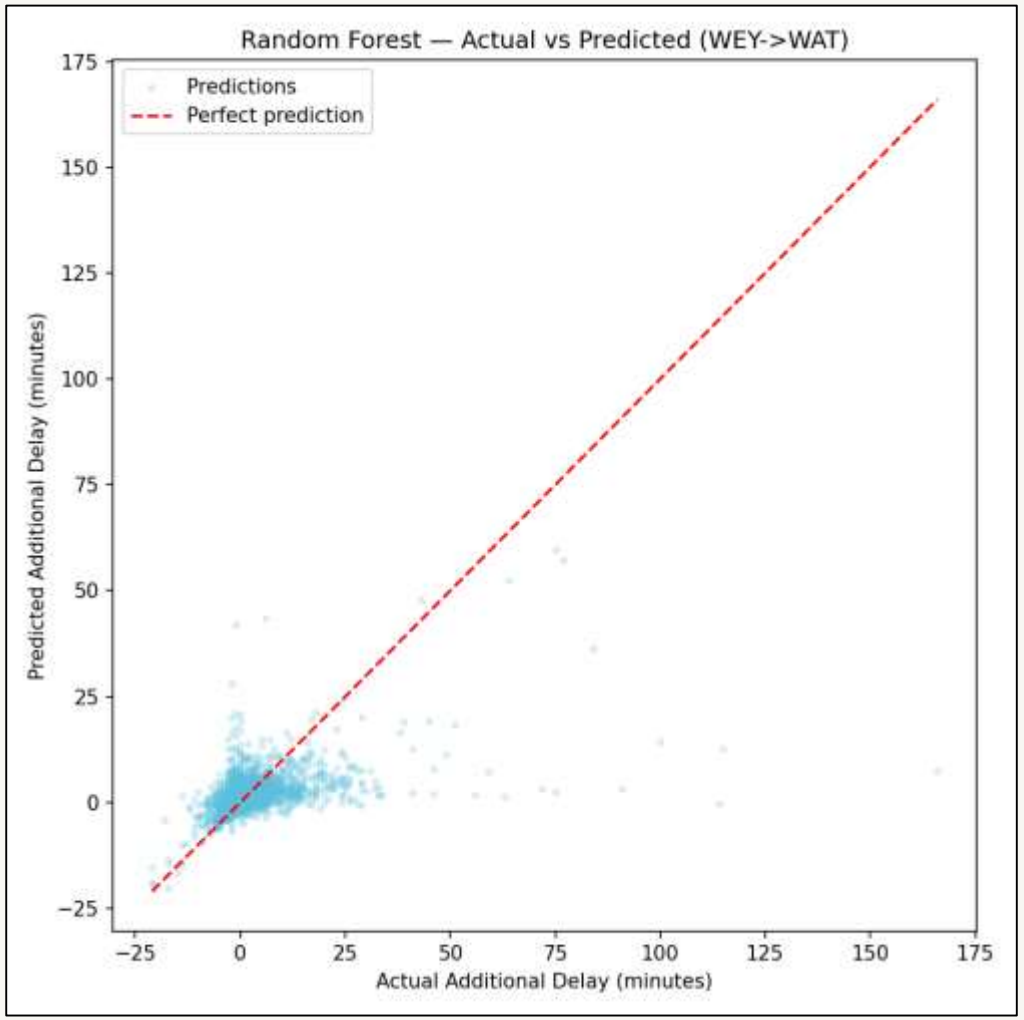


Feature Importance



Random Forest - Actual vs Predicted (WAT <-> WEY)





Chatbot Integration

- ➔ Chatbot collects 6 slots through conversation:
  - ◆ Direction (WAT2WEY or WEY2WAT)
  - ◆ Origin station
  - ◆ Destination station
  - ◆ Planned arrival time (HH:MM)
  - ◆ Current station
  - ◆ Current delay (minutes)
- ➔ Input validated at each step
- ➔ Total delay = current delay + predicted additional delay

Task 3:

Railway Disruption Expert System

How the railway chatbot retrieves focused contingency guidance for disruption scenarios.

Railway Chatbot / Staff Contingency Plan Assistant

CENTRAL PURPOSE

Task 3 transforms the chatbot into an explainable assistant for railway disruption planning.

- It supports line blockage and station disruption enquiries.
- It returns selected operational guidance for staff and passenger needs.
- It retrieves existing expert knowledge; it does not predict delays or control live services.

What is an expert system?

Task 3 is built around stored domain knowledge and reasoning logic, rather than prediction.

An expert system is an AI system that uses specialist knowledge, facts and decision rules to provide advice similar to a human expert.

Its value is explainability: the recommendation can be linked back to stored guidance and the information supplied by the user.

In this project, the expert domain is railway disruption response.

HOW THIS APPLIES TO TASK 3

- Knowledge base: contingency plans and station disruption guidance provide the stored operational expertise.
- Reasoning: the incident type, location, blockage condition and required audience determine which guidance is returned.
- Interface: a guided chatbot conversation collects missing incident details before presenting advice.
- Difference from ML: Task 3 selects guidance; it is not trained to predict disruption outcomes.

Why expert system is needed ??

Railway disruption decisions depend on finding the right instruction quickly and consistently.

THE OPERATIONAL PROBLEM

- Disruption guidance is distributed through line contingency plans and station disruption documents, with different information for different roles.
- Manual document searching can be slow during an incident and may lead to inconsistent retrieval of relevant advice.
- Users may know the incident location and need, but not which source document contains the correct procedure.

TASK 3 SCOPE

Two supported disruption scenarios

- Line blockage: the assistant collects an affected line section and whether the blockage is partial or full.
- Station disruption: the assistant collects the affected station and the required type of guidance.
- Purpose: provide a faster guided route to focused existing contingency advice, not create new railway rules.

How expert knowledge was prepared

The system turns operational documents into structured records and searchable semantic rule chunks.

KNOWLEDGE SOURCE AND STORED USE

Source material	Content captured	How it uses
Line contingency plans	Route sections, blockage type and role-specific advice	SQLite supports exact plan selection for line blockage requests.
Station disruption plans	Local communications, Control contact and station operation tips	Station-specific answers are returned for staff or passenger need.
Extracted rule chunks	Searchable textual knowledge prepared during ingestion	ChromaDB can support natural-language retrieval when structured fields are incomplete.

VERIFIED IMPLEMENTATION SCALE

72
line contingency plans

195
station disruption plans

1,905
station tips

2299
knowledge files

3496
ChromaDB chunks

What expert system adds to the chatbot

A guided conversation gathers enough incident detail before the expert system returns advice.

INCIDENT WORKFLOW 1 | LINE BLOCKAGE

- The assistant asks which two stations or locations define the affected line section.
- It requests confirmation of the location, then asks whether the blockage is partial or full.
- Once the blockage condition is confirmed, it asks which type of operational advice is needed.

Example: both lines blocked between Woking and Guildford.

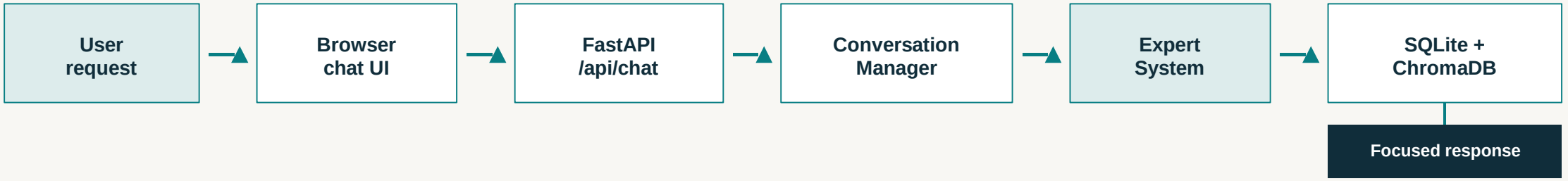
INCIDENT WORKFLOW 2 | STATION DISRUPTION

- The assistant asks which station is affected, then confirms that station before searching for advice.
- Users can request station staff advice, signaller information, passenger information, service alterations, alternative journeys or all guidance.
- The final response is audience-focused, so users are not forced to read all stored material for every incident.

Example: station staff advice for disruption at Bournemouth.

How the expert system works

IMPLEMENTED TECHNICAL FLOW



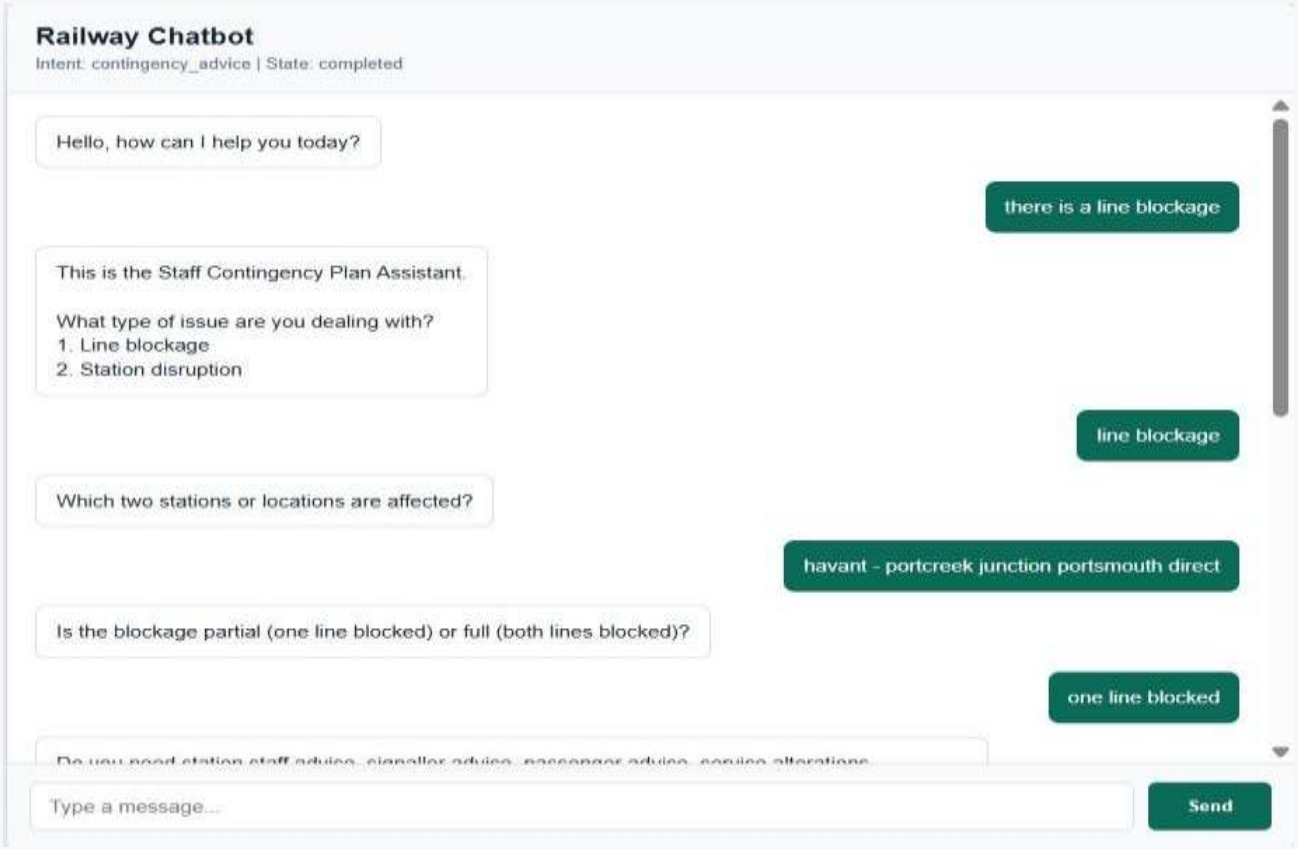
WHAT HAPPENS INSIDE THIS FLOW

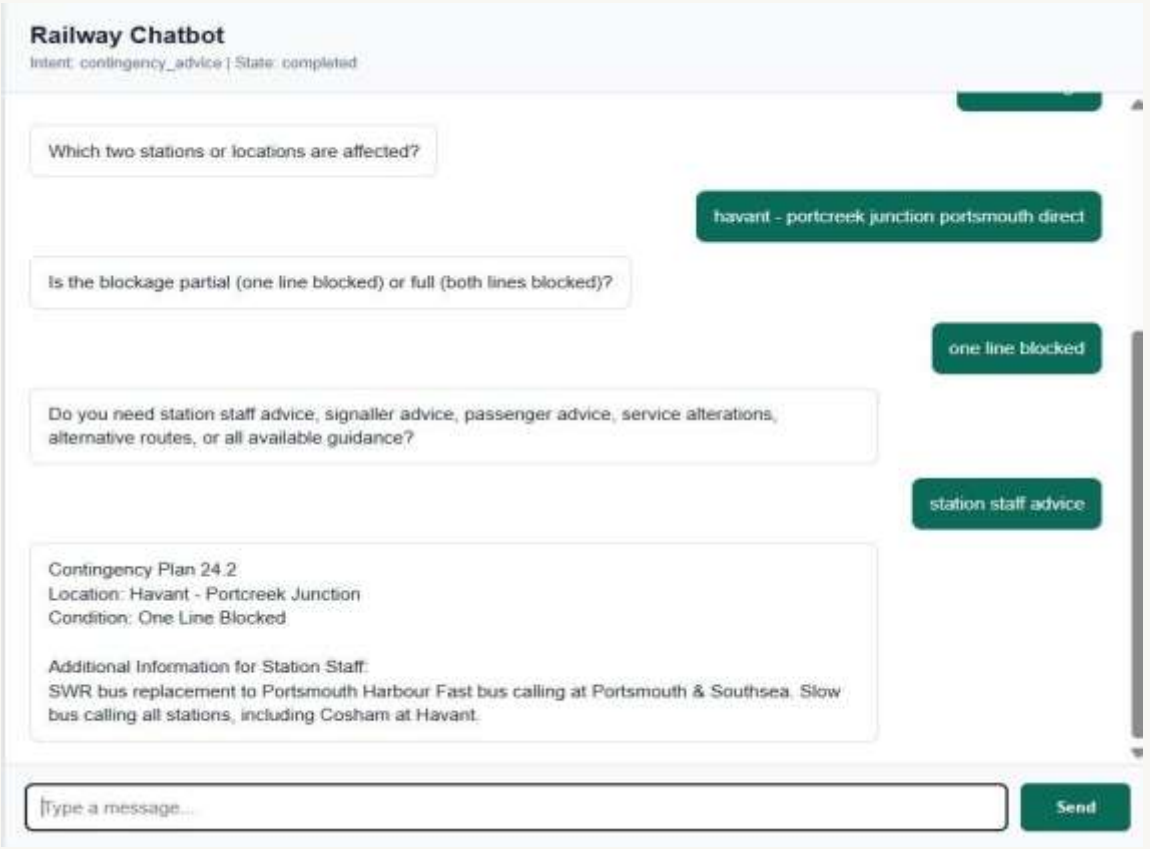
1. The conversation manager stores the incident slots and asks follow-up questions when required information is missing.

2. SQLite selects a structured contingency or station plan when location and incident details match.

3. ChromaDB semantic retrieval can supplement missing fields or return relevant operational guidance when exact matching is insufficient.

OUTPUT


Railway Chatbot
Intent: contingency_advice | State: completed
Hello, how can I help you today?
there is a line blockage
This is the Staff Contingency Plan Assistant.
What type of issue are you dealing with?
1. Line blockage
2. Station disruption
line blockage
Which two stations or locations are affected?
havant - portcreek junction portsmouth direct
Is the blockage partial (one line blocked) or full (both lines blocked)?
one line blocked
Type a message...
Send


Railway Chatbot
Intent: contingency_advice | State: completed
Which two stations or locations are affected?
havant - portcreek junction portsmouth direct
Is the blockage partial (one line blocked) or full (both lines blocked)?
one line blocked
Do you need station staff advice, signaller advice, passenger advice, service alterations, alternative routes, or all available guidance?
station staff advice
Contingency Plan 24.2
Location: Havant - Portcreek Junction
Condition: One Line Blocked
Additional Information for Station Staff:
SWR bus replacement to Portsmouth Harbour Fast bus calling at Portsmouth & Southsea. Slow bus calling all stations, including Cosham at Havant.
Type a message...
Send

What Task 3 achieves - and what remains

An implemented and explainable disruption-assistance feature within the wider railway chatbot.

CONCLUSION

Task 3 changes the chatbot from a general railway information interface into an expert-system assistant for disruption planning.

- It guides users to relevant existing advice for line blockages and station disruptions.
- It combines structured matching with semantic support while keeping the response clear and audience-focused.

LIMITATIONS AND IMPROVEMENTS

- Quality depends on the accuracy and completeness of the extracted contingency documents.
- Unusual station spelling or route wording can reduce matching reliability; stronger alias normalisation is a sensible improvement.
- Regression tests and an admin evidence view would make retrieval behaviour easier to validate.
- The current system has no live railway-control connection; live integration would be future work, not a present claim.

CONCLUSION

- We successfully developed a railway ticket chatbot that supports a realistic user conversation. The system collects all required journey information, handles railcards and return tickets, validates station names, and uses National Rail API data where available.
- The development process involved solving practical chatbot issues such as incomplete API data, ambiguous station names, and railcard parsing. Overall, the chatbot provides a useful and interactive way for users to search for train tickets.
- We developed a train delay prediction system for users to predict when their delayed train will arrive at their destination.
- We successfully developed an expert-system chatbot for railway disruption guidance. It collects key incident details through conversation and returns clear operational advice.
- We successfully developed a hybrid retrieval approach using SQLite and ChromaDB. This helps the chatbot match structured disruption details while also handling natural user wording.

THANK YOU